C++ Systems Programming — Week 2 Notes

Object Lifetime, RAII, Copy/Move Semantics, Smart Pointers, and Value Categories

Systems Programming Course Notes

October 11, 2026

Abstract

These notes cover the Week 2 topics in a connected order. The central theme is **resource ownership**: which object owns a resource, how ownership is copied or transferred, and how C++ guarantees that resources are released safely.

1 The Big Picture: Objects and Resources

A C++ object may own a resource such as:

- Dynamically allocated memory
- A file handle
- A socket
- A mutex lock
- A database connection
- An operating-system handle

Resources have lifetimes. If a resource is acquired but never released, the program may leak it. If two owners release the same resource independently, the program may double-free it or otherwise exhibit undefined behavior. Modern C++ addresses these problems through **RAII**, value semantics, and smart pointers.

1.1 Automatic storage versus dynamic storage

```
void example() {
    int x = 10;                 // Local object
    int* p = new int(20);       // Dynamically allocated object

    delete p;                   // Must release this allocation
}
```

`x` is automatically destroyed when its scope ends. The integer created by `new` remains allocated until it is deleted or its ownership is otherwise managed. Raw `new` and `delete` are useful for understanding the model, but ordinary application code should generally prefer standard resource-managing types such as `std::vector`, `std::string`, and smart pointers.

1.2 Lifetime and scope

An object's **scope** concerns where its name is visible. Its **lifetime** is the period during which the object exists. These concepts are related but not identical.

```
int* make_pointer() {
    int local = 42;
    return &local;  // Wrong: returns the address of a local object
}
```

After `make_pointer()` returns, `local`'s lifetime has ended. The returned pointer dangles. Dereferencing it has undefined behavior. A pointer can remain in scope after the object it points to has died. Therefore, the existence of a pointer does not guarantee the existence of its pointee.

2 Constructors, Destructors, and Object Lifetime

2.1 Constructor

A constructor initializes an object when it is created.

```
#include <string>

class Student {
public:
    std::string name;

    explicit Student(const std::string& n)
        : name(n) {}
};
```

The initializer list `: name(n)` initializes the member directly. This is generally preferable to default-constructing a member and then assigning to it in the constructor body.

2.2 Destructor

A destructor runs when an object's lifetime ends. It is written with a tilde before the class name.

```
#include <iostream>

class Example {
public:
    Example() { std::cout << "Construct\n"; }
    ~Example() { std::cout << "Destruct\n"; }
};

int main() { Example e; }
```

Output:

```
Construct
Destruct
```

A destructor is commonly responsible for releasing resources owned by an object. If the object contains standard resource-managing members, those members release their own resources automatically.

2.3 Destruction order

For local automatic objects in the same scope, destruction occurs in reverse order of completed construction.

```
#include <iostream>
struct A { A() { std::cout << "A+\n"; } ~A() { std::cout << "A-\n"; } };
struct B { B() { std::cout << "B+\n"; } ~B() { std::cout << "B-\n"; } };
int main() { A a; B b; }
```

Output:

```
A+
B+
B-
A-
```

2.4 Exception stack unwinding

When an exception exits a scope, fully constructed automatic objects in that scope are destroyed while the stack unwinds.

```
#include <iostream>
#include <stdexcept>

struct Guard {
    ~Guard() { std::cout << "Guard␣released\n"; }
};

int main() {
    try {
        Guard g;
        throw std::runtime_error("failure");
    } catch (const std::exception& e) {
        std::cout << "Caught:␣" << e.what() << '\n';
    }
}
```

Typical output:

```
Guard released
Caught: failure
```

3 RAII — Resource Acquisition Is Initialization

RAII ties resource ownership to an object's lifetime:

1. Acquire a resource while constructing an owner.
2. Keep the resource under that object's control.
3. Release the resource when the owner is destroyed.

For example, `std::ofstream` closes its file automatically when the stream object is destroyed.

```cpp
#include <fstream>
#include <stdexcept>

void write_report() {
    std::ofstream out("report.txt");
    if (!out) {
        throw std::runtime_error("Could not open report.txt");
    }
    out << "Report contents\n";
} // The stream is destroyed and the file is closed here.
```

3.1 Why RAII is safer than manual cleanup

Manual cleanup is fragile:

```cpp
void fragile() {
    auto* p = new int[100];
    perform_operation();  // Suppose this throws
    delete[] p;           // May never execute
}
```

If `perform_operation()` throws, the `delete[]` line is skipped and the allocation leaks. A standard container handles the resource through RAII safely:

```cpp
#include <vector>
void safer() {
    std::vector<int> values(100);
    perform_operation();
} // The vector releases its allocation on every normal exit path.
```

3.2 RAII applies beyond memory

RAII can manage file streams, locks through `std::lock_guard` or `std::unique_lock`, smart pointers, scoped transactions, and other resources represented by an object with a correct destructor.

4 Raw Pointers and Ownership

A raw pointer such as `int*` stores an address. It does not, by itself, tell us whether it owns the pointed-to object.

```cpp
int value = 5;
int* observer = &value;  // Non-owning pointer
```

Useful distinctions:

- **Owning pointer:** responsible for the lifetime of the pointed-to resource.
- **Non-owning pointer:** observes or accesses a resource owned elsewhere.
- **Dangling pointer:** points to an object whose lifetime has ended.
- **Memory leak:** allocated or acquired resource is no longer reachable for proper release.
- **Double deletion:** the same allocation is released more than once; this is undefined behavior.

5 Shallow Copy versus Deep Copy

Suppose a class owns a dynamically allocated integer.

```
class Buffer {
public:
    int* data;
    explicit Buffer(int value) : data(new int(value)) {}
    ~Buffer() { delete data; }
};
```

5.1 The shallow-copy problem

If the compiler-generated copy constructor is used, it copies the pointer value. Both objects now point to the same allocation, resulting in undefined behavior upon double destruction.

5.2 Deep copy

A deep copy allocates separate storage and copies the value or elements.

```
class Buffer {
public:
    int* data;
    explicit Buffer(int value) : data(new int(value)) {}
    Buffer(const Buffer& other) : data(new int(*other.data)) {}
    ~Buffer() { delete data; }
};
```

6 Copy Constructor versus Copy Assignment

6.1 Copy construction

```
Buffer a(10);
Buffer b = a;   // Copy constructor
```

6.2 Copy assignment

```
Buffer a(10);
Buffer b(20);
b = a;          // Copy-assignment operator
```

A safe assignment implementation checks for self-assignment via `this != &other`:

```
Buffer& operator=(const Buffer& other) {
    if (this != &other) {
        *data = *other.data;
    }
    return *this;
}
```

7 The Rule of Three, Five, and Zero

- **The Rule of Three:** If a class needs a custom destructor, copy constructor, or copy-assignment operator, it likely needs all three.
- **The Rule of Five:** Expands the Rule of Three to include move constructor and move-assignment operator.
- **The Rule of Zero:** Design classes so that you do not need to write custom destructors, copy operations, or move operations by using standard resource-managing members like `std::vector` or `std::string`.

8 Exception Safety and Copy-and-Swap

The **strong exception guarantee** ensures that if an operation fails by throwing, the observable state of the target object remains unchanged. Copy-and-swap is a common way to achieve this.

```
#include <algorithm>
#include <cstddef>
#include <utility>

class Buffer {
private:
    std::size_t size_ = 0;
    int* data_ = nullptr;
public:
    explicit Buffer(std::size_t n) : size_(n), data_(n ? new int[n]{} :
       nullptr) {}
    ~Buffer() { delete[] data_; }

    Buffer(const Buffer& other)
        : size_(other.size_), data_(other.size_ ? new int[other.size_] :
           nullptr) {
        if (size_ != 0) {
            std::copy(other.data_, other.data_ + size_, data_);
        }
    }

    void swap(Buffer& other) noexcept {
        using std::swap;
        swap(size_, other.size_);
        swap(data_, other.data_);
    }

    Buffer& operator=(Buffer other) {
        swap(other);
        return *this;
    }
};
```

9 Move Semantics

Move semantics allow a type to transfer a resource from one object to another rather than duplicate the resource via rvalue references (`&&`).

```
Buffer(Buffer&& other) noexcept
    : size_(other.size_), data_(other.data_) {
    other.size_ = 0;
    other.data_ = nullptr;
}
```

std::move is a cast-like utility that allows an expression to be treated as an rvalue for overload resolution, signaling that moving from the object is appropriate.

10 Smart Pointers

Standard-library objects managing pointer ownership through RAII:

- **std::unique_ptr:** Exclusive ownership. Non-copyable, movable via `std::move`.
- **std::shared_ptr:** Shared ownership via reference counting.
- **std::weak_ptr:** Non-owning observation of a `std::shared_ptr` target, preventing circular references.

Table 1: Smart Pointer Summary

Property	unique_ptr	shared_ptr	weak_ptr
Ownership	Exclusive	Shared	None
Keeps object alive	Yes	Yes	No
Copyable	No	Yes	Yes
Typical role	Primary owner	Multiple owners	Non-owning observer

11 Value Categories and Perfect Forwarding

C++ expressions have value categories (lvalues, prvalues, xvalues). **Perfect forwarding** uses forwarding references (`T&&`) alongside `std::forward<T>` in template wrappers to preserve the caller's value category.

```
template <typename T>
void wrapper(T&& x) {
    process(std::forward<T>(x));
}
```

12 Summary of Best Practices

1. Make ownership explicit.
2. Use RAII for resource management.
3. Prefer the Rule of Zero; use the Rule of Five only when writing low-level resource wrappers.
4. Choose `std::unique_ptr` as the default smart pointer.